SUJET

Optimiser les tournées de collecte du dernier kilomètre grâce au Machine Learning

COURS : OPTIMISATION POUR L'APPRENTISSAGE AUTOMATIQUE
MASTER AIDA — DAUPHINE-PSL



Réalisé Par : BEN MUSTAPHA Siwar

Encadré Par : Mr. Ayadi Slim

Année Universitaire 2026 – 2027

Sommaire

1. Introduction et problématique 3
Contexte 3
Problème métier 3
Objectifs du projet 3
Intérêt du Machine Learning 3
KPI suivis et stratégies comparées 3
2. Dataset et preprocessing 4
Variables importantes 4
Qualité des données et traitements réalisés 4
3. Exploratory Data Analysis 5
4. Machine Learning 6
5. Optimisation et conclusion 7
Références 7

Optimiser les tournées de collecte du dernier kilomètre grâce au Machine Learning

Tournées dynamiques avec fenêtres horaires — dataset LaDe-P (Cainiao), ville de Yantai

Enseignant : Mr Slim Ayedi

1. Introduction et problématique

Contexte

Le **dernier kilomètre** représente une part majeure du coût logistique. Chaque jour, un coursier collecte des colis chez des clients qui ont reçu une **fenêtre horaire de 2 heures**. Les demandes arrivent tout au long de la journée : le coursier ne connaît pas toutes ses tâches le matin.

Problème métier

À chaque fois qu'il termine une collecte, le coursier doit décider **où aller ensuite**. Un mauvais ordre de visite allonge la distance, consomme du temps de travail et fait manquer des échéances. Il s'agit d'une version **dynamique** du problème du voyageur de commerce avec fenêtres horaires (**TSPTW**), un problème d'optimisation combinatoire NP-difficile.

Objectifs du projet

- Préparer et analyser les données réelles de 1 146 781 colis collectés par 2 593 coursiers sur 184 jours.
- Apprendre par Machine Learning le **temps nécessaire entre deux visites** (trajet + service).
- Utiliser ces prédictions dans un **modèle d'optimisation** qui choisit l'ordre de visite à chaque arrêt.
- Mesurer honnêtement le gain par rapport au **coursier réel** et à une **optimisation sans ML**.

Intérêt du Machine Learning

Un optimiseur a besoin d'une **matrice de temps** entre tous les points. Ce temps n'existe pas dans les données : il dépend de la distance, du quartier, de l'heure ou du type de lieu. Le ML l'estime à partir de près d'un million de trajets observés. **Le ML prédit, l'optimisation décide.**

KPI suivis et stratégies comparées

KPI	Définition	Unité
Distance parcourue	Somme des distances entre visites successives (calcul exact, sans modèle)	km / journée
Temps de déplacement	Somme des temps de trajet et de service	min / journée
Durée de la tournée	De la première à la dernière collecte (attente de nouvelles tâches incluse)	min / journée
Collectes en retard	Visites réalisées après la fin de leur fenêtre	visites / journée

Stratégie	Qui décide de l'ordre ?	Temps utilisés pour décider
Réel (coursier)	Le coursier, tel qu'enregistré dans les données	Son expérience
Scénario 1 — Optimisé sans ML	Heuristique d'optimisation à chaque arrêt	Formule : service médian + distance ÷ vitesse médiane
Scénario 2 — Optimisé avec ML	La même heuristique, à chaque arrêt	Matrice horaire prédite par le Gradient Boosting

Comparer les scénarios 1 et 2 isole la **valeur ajoutée du ML** ; les comparer au réel mesure la **valeur ajoutée de l'optimisation**.

2. Dataset et preprocessing

Source : LaDe (Last-mile Delivery), premier grand dataset public de livraison du dernier kilomètre, publié par Cainiao, la filiale logistique d'Alibaba (Wu et al., 2023 ; licence Apache-2.0). Nous utilisons le sous-ensemble **collecte** de la ville de **Yantai** (fichier pickup_yt.csv).

Caractéristique	Valeur	Caractéristique	Valeur
Lignes (colis)	1 146 781	Colonnes	19
Coursiers	2 593	Journées-coursier	71 521
Régions / AOI	30 / 6 024	Période	mai → octobre (184 jours)
Fenêtre médiane	120 min	Collectes en retard (réel)	0,47 %

Variables importantes

Coordonnées du point de collecte (lng, lat), région et aire d'intérêt (region_id, aoi_id, aoi_type), coursier, fenêtre horaire (time_window_start / end), moment où la tâche est connue (**accept_time**) et moment réel de la collecte (**pickup_time**).

Qualité des données et traitements réalisés

Étape	Lignes	Justification
Doublons / identifiants répétés	0 / 0	Aucun doublon : rien à supprimer
Incohérences temporelles	0	Aucune collecte avant l'acceptation, aucune fenêtre inversée
Points hors zone	0	Coordonnées toutes plausibles
Colonnes GPS manquantes	21 % et 36 %	Non utilisées : les coordonnées déclarées sont complètes
Regroupement en visites	1 135 167	Colis consécutifs au même lieu = une seule visite
Paires visite → suivante	1 063 646	Unité d'apprentissage du ML
Filtre de causalité	976 050	La visite suivante devait être connue au départ (sinon on mesure une attente, pas un trajet)
Pauses exclues de l'apprentissage	16 547	Écarts > 120 min (1,7 %) : pauses ou autres activités ; conservés dans les tournées réelles
Base d'apprentissage finale	959 503	9 variables explicatives connues avant le départ

Dates : l'année n'est pas fournie ; une année fictive sert uniquement aux calculs, et le jour de la semaine n'est donc pas utilisé. **Découpage sans fuite de données :** apprentissage sur mai–août, test sur septembre–octobre. Les coursiers sont tirés au hasard en deux groupes disjoints : le groupe A entraîne le **modèle planificateur**, le groupe B entraîne un **modèle évaluateur** indépendant qui sert seulement à noter les tournées.

Figure 3 — Temps entre deux visites consécutives

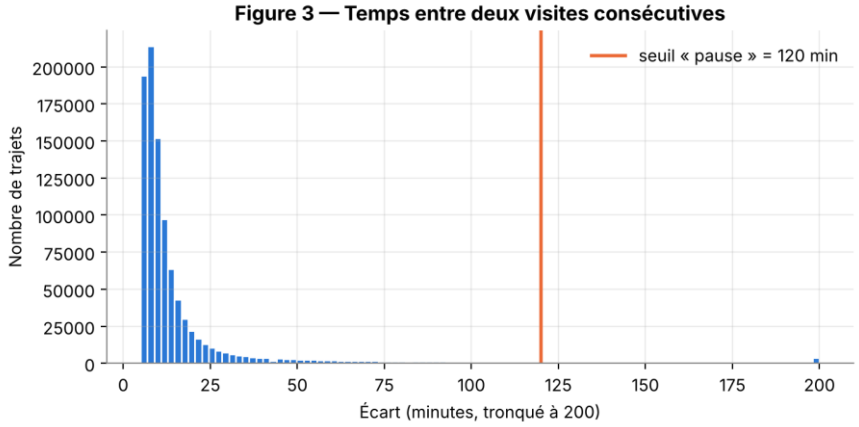


Figure 3 — Distribution du temps entre deux visites consécutives : médiane de 9 min, longue queue au-delà du seuil de pause (120 min).

3. Exploratory Data Analysis

Les figures portent la même numérotation que dans le Notebook. L'EDA répond à trois questions qui orientent directement la modélisation : **de quoi dépend le temps ?**, **les fenêtres sont-elles contraignantes ?** et **quand les tâches sont-elles connues ?**

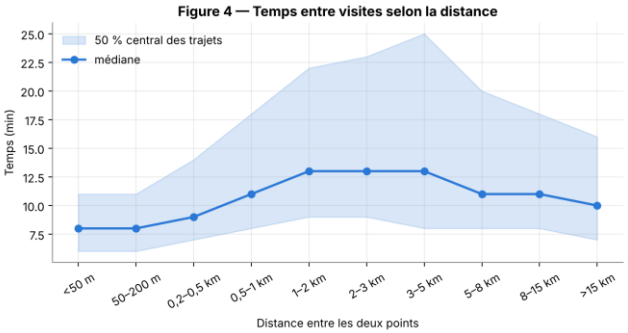


Figure 4 — Temps entre visites selon la distance (médiane et 50 % central).

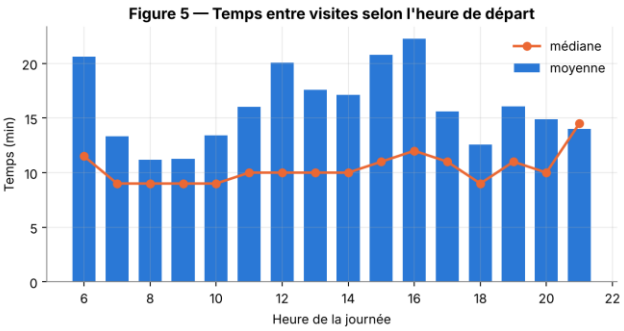


Figure 5 — Temps entre visites selon l'heure de départ.

Le temps n'est pas proportionnel à la distance. Il passe de 8 min (moins de 50 m) à 13 min (1 à 5 km), puis baisse pour les longs trajets, probablement réalisés en véhicule. À distance égale, la dispersion est très forte. **L'heure compte :** le temps moyen est d'environ 11 à 13 min le matin et monte jusqu'à environ 20 min l'après-midi. La matrice de temps doit donc **dépendre de l'heure**, ce qu'une formule fixe ne permet pas.

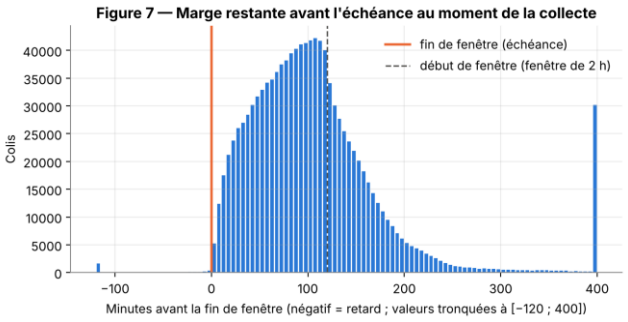


Figure 7 — Marge restante avant l'échéance au moment de la collecte.

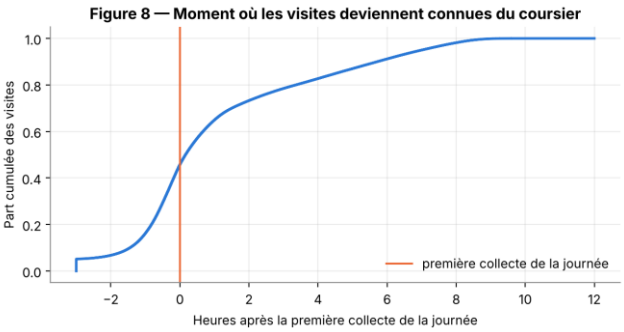


Figure 8 — Moment où les visites deviennent connues du coursier.

Les coursiers respectent déjà presque toujours les échéances : 0,47 % de retards seulement, et 32 % des colis sont collectés avant le début de la fenêtre. La fin de fenêtre est donc une **échéance**, le début n'est pas contraignant. Le gain attendu porte sur le **temps et la distance**, sans dégrader le service.

Seulement 46 % des visites sont connues avant la première collecte. Planifier toute la journée le matin serait irréaliste, car ce serait utiliser des informations futures. Nous optimisons donc **dynamiquement** : à chaque arrêt, parmi les seules tâches déjà connues.

Corrélations (Spearman) : la distance (0,27) et l'heure (0,17) sont les variables les plus liées au temps, mais toutes les corrélations restent faibles. Aucune variable seule ne suffit : c'est un argument pour un modèle non linéaire, et un signe que la prédiction sera difficile.

4. Machine Learning

Cible : temps (min) entre la fin d'une visite et le début de la suivante. **Variables :** distance, même AOI, types d'AOI de départ et d'arrivée, heure de départ, colis à l'arrivée, région, latitude et longitude d'arrivée. Toutes sont connues avant de partir. **Entraînement :** 327 693 paires (coursiers A, mai–août) ; **test :** 153 772 paires (coursiers A, sept.–oct.).

Modèles comparés, du plus simple au plus flexible : une **baseline** à base de moyennes historiques (service médian de 8 min + distance ÷ 16,7 km/h), une **régression linéaire** (explicable), un **arbre de décision**, une **Random Forest** et un **Gradient Boosting** (scikit-learn).

Modèle	MAE (min)	Biais moyen (min)	RMSE (min)	R^2
Baseline (moyennes historiques)	7,98	−5,07	17,78	-0,104
Régression linéaire	9,39	−0,13	16,55	0,043
Arbre de décision	9,24	−0,07	16,25	0,078
Random Forest	9,11	−0,06	16,04	0,101
Gradient Boosting (retenu)	9,01	−0,04	15,92	0,115

Tableau — Performances sur le jeu de test (sept.–oct.). Biais = moyenne (prédiction − réalité).

Figure 11 — Comparaison des modèles sur le jeu de test (sept.–oct.)

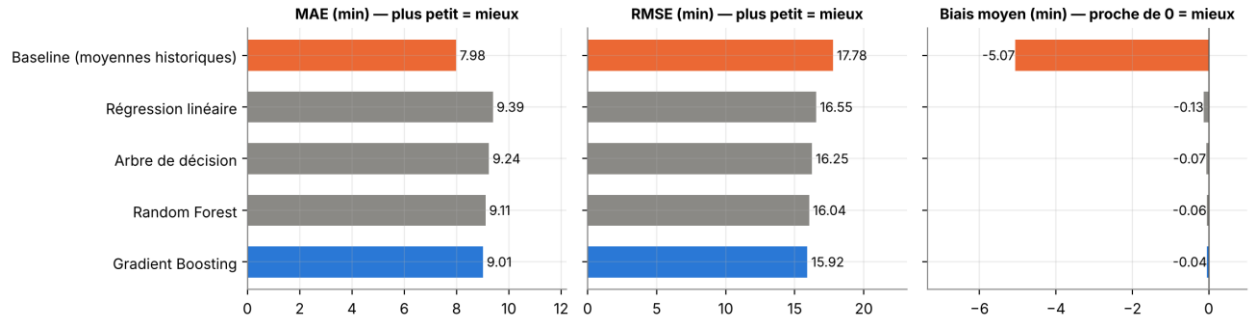


Figure 11 — Comparaison des modèles : MAE, RMSE et biais moyen.

Un résultat contre-intuitif et instructif. La baseline a la plus petite MAE (7,98 min) car elle prédit le trajet « typique ». Mais elle **sous-estime en moyenne chaque trajet de 5,1 min**, ce qui fait plus d'une heure d'erreur sur une tournée de 20 trajets. Le Gradient Boosting a une MAE un peu plus élevée (9,01 min) mais la **meilleure RMSE** (15,92), le meilleur R^2 (0,11) et un **biais quasi nul** (-0,04 min). Il évite surtout les grosses erreurs sur les longs trajets (Figure 12).

Choix du modèle : une tournée additionne les temps de ses trajets. Pour planifier, il faut donc des temps **justes en moyenne**, et pas seulement « souvent proches ». Nous retenons le Gradient Boosting, meilleur en RMSE et sans biais, et rapide à entraîner (6 s). Le R^2 reste modeste : une grande part du temps dépend d'éléments absents des données (attente du client, appel, livraisons intercalées).

Figure 12 — Erreur moyenne selon la distance du trajet

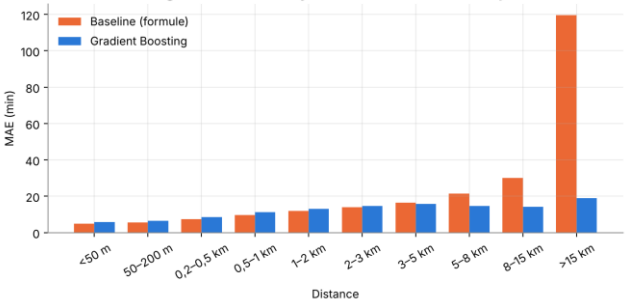


Figure 12 — Erreur moyenne selon la distance du trajet.

Figure 13 — Importance des variables (Gradient Boosting)

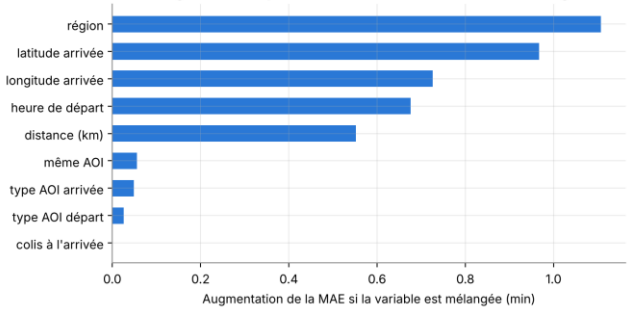


Figure 13 — Importance des variables (permutation).

Ce que le modèle a appris : la localisation (région, coordonnées) et l'heure pèsent plus que la distance. Certains quartiers et l'après-midi sont plus lents. Le type d'AOI compte peu.

5. Optimisation et conclusion

Modèle mathématique. À l'instant τ, le coursier connaît N visites en attente. On minimise **Σ t(i,j)·x(i,j) + λ·Σ L(j)**, où x(i,j) ∈ {0,1} vaut 1 si l'on va de i à j, t(i,j) est le **temps prédit par le ML** pour l'heure de départ, et L(j) ≥ a(j) − l(j) est le retard sur l'échéance l(j). Les contraintes imposent une entrée et une sortie par visite, ainsi que la propagation du temps a(j) ≥ a(i) + t(i,j) − M·(1 − x(i,j)), qui élimine aussi les sous-tours. C'est un **MILP de type TSPTW**, avec λ = 5.

Résolution dynamique. À chaque arrêt, une heuristique (construction gloutonne + **2-opt** + **or-opt**) réordonne les visites connues ; le coursier part vers la première. Validée contre l'**optimum exact (solveur HiGHS)** sur 30 instances de 8 visites, elle trouve l'optimum dans 83 % des cas, avec un écart moyen de 0,17 %, en 1,0 ms contre 0,40 s pour le MILP. **Évaluation :** 150 journées de test ; les temps réalisés sont donnés par l'évaluateur indépendant, dont les durées simulées sont corrélées à 0,61 avec les durées réelles (erreur médiane 9 %). La distance est calculée exactement.

KPI moyen par journée	Réel (coursier)	Optimisé sans ML	Optimisé avec ML	Avec ML vs réel	Avec ML vs sans ML
Distance (km)	10,45	8,23	8,93	−14,6 %	+8,5 %
Temps de déplacement (min)	318,5	295,7	298,6	−6,2 %	+1,0 %
Durée de la tournée (min)	462,9	430,4	431,4	−6,8 %	+0,3 %
Collectes en retard	1,05	1,39	0,92	−12 %	−34 %
Minutes de retard	45,2	42,4	30,7	−32 %	−28 %

Figure 15 — KPI moyens par journée (150 journées de test, λ = 5)

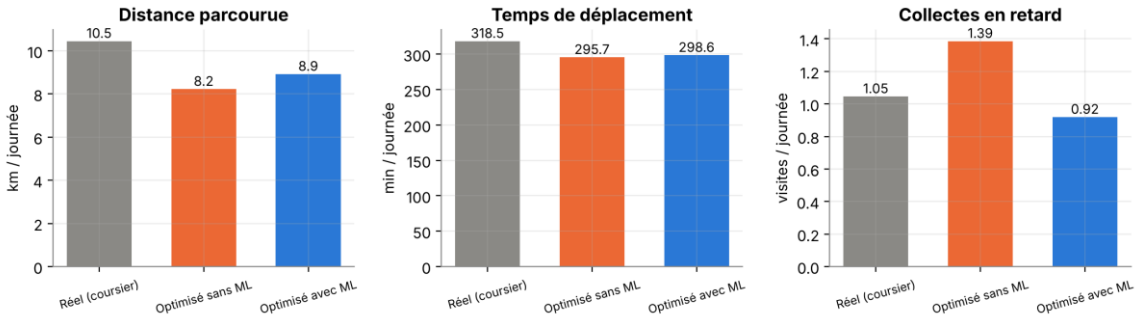


Figure 15 — KPI moyens par journée pour les trois stratégies (150 journées de test, λ = 5).

Gains simulés. L'optimisation réduit nettement les tournées par rapport au terrain : −14,6 % de distance et −6,2 % de temps de déplacement avec ML. Elle est meilleure que le coursier réel dans 82 % des journées (Wilcoxon, $p < 0{,}001$). **Sans ML**, le gain kilométrique est plus fort (−21,3 %) mais se paie par **plus de retards** que l'ordre réel, car la formule sous-estime les temps. **Avec ML**, les retards baissent de 34 % par rapport au scénario sans ML, pour +1,0 % de temps de déplacement. L'analyse de sensibilité (λ de 0 à 20, voir Notebook) montre que la formule ne corrige pas ses retards même avec une forte pénalité : **une bonne pénalité ne compense pas une mauvaise prédiction.**

Limites. Les gains de temps et de retard sont **simulés** (une tournée jamais réalisée ne peut pas être observée) ; l'évaluateur surestime les retards en valeur absolue (1,05 simulés contre 0,08 observés par journée), donc seules les comparaisons entre stratégies sont interprétables. Les distances sont à vol d'oiseau, le coursier réel dispose d'informations absentes des données, et le R^2 du modèle reste faible.

Conclusion. Le projet relie une prédiction ML à une décision opérationnelle mesurée. L'optimisation raccourcit les tournées ; le ML rend le plan **fiable** en préservant le respect des échéances. Pistes : distances routières (roads.csv), version multi-coursiers (VRPTW), prévision des futures demandes.

Références

Wu, L. et al. (2023). LaDe: The First Comprehensive Last-mile Delivery Dataset from Industry. arXiv:2306.10675.
Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
Huangfu, Q. & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. Mathematical Programming Computation, 10, 119–142 (solveur HiGHS).
Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. Operations Research, 35(2), 254–265.
Pillac, V., Gendreau, M., Guéret, C. & Medaglia, A. L. (2013). A review of dynamic vehicle routing problems. European Journal of Operational Research, 225(1), 1–11.